MLOPS II, MIA, FIUBA

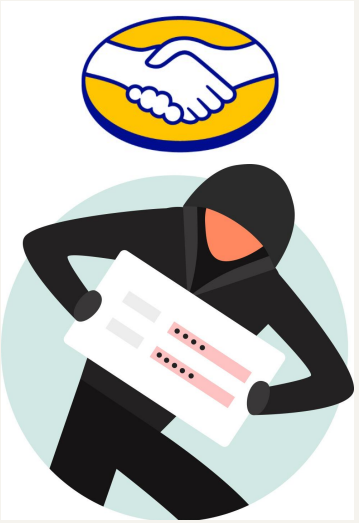
Detección de fraude

Florencia Priscilla Vela

Trabajo integrador

EL PROBLEMA

El fraude aprende a esquivar al modelo.



Los defraudadores cambian su comportamiento justamente para escapar de la detección. Un modelo que hoy funciona, mañana se degrada.

> **El valor no está en entrenar un buen modelo, sino en poder reemplazarlo de forma segura y continua.**

La demo también la pongo en screenshots...
just in case



Lo que condiciona todo el diseño

~1 %

Desbalance extremo

Casi todo es legítimo: se mide con PR-AUC y recall, no con accuracy.

ms

Baja latencia

Aprobar o retener es una decisión online, dentro del flujo de pago.

días

El label llega tarde

La verdad aparece después: contracargos y revisión manual.

Dataset: pointe77/credit-card-transaction, ~1.85 M transacciones simuladas, una fila por transacción

LA PREGUNTA CENTRAL

¿Cómo pongo en producción un modelo v2 si todavía no sé si captura más fraude que el v1?

OPCIONES DE DESPLIEGUE

Tres formas de convivir con dos modelos

A

Doble modelo con fallback

v1 y v2 a la vez: el pago pasa solo si ninguno marca fraude.

Pero: suma fricción.

B

Split de tráfico / canary

Un X % del tráfico va al v2 y se amplía de a poco.

Pero: clientes reales decididos por un modelo no validado.

C (ELEGIDA)

Despliegue en sombra

El v2 ve todo el tráfico real, predice y guarda, pero no decide.

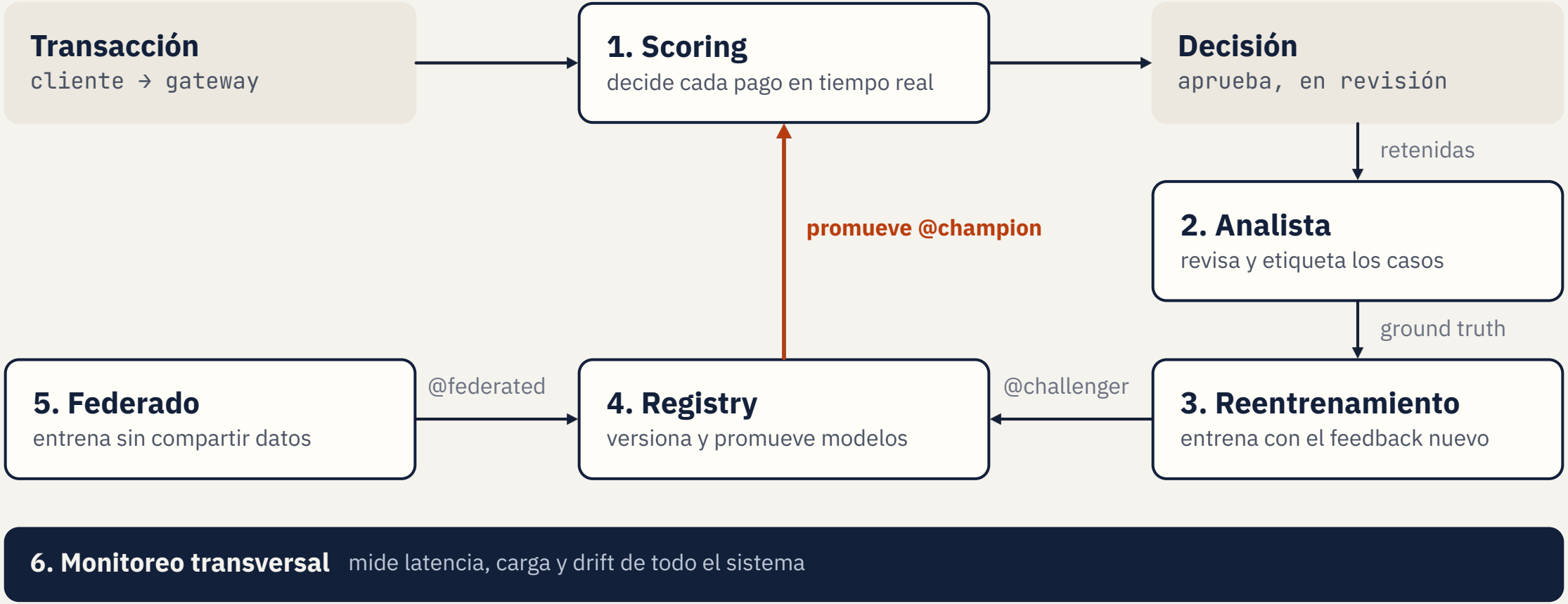
Gana: evidencia contra la ground truth, sin riesgo.

LA SOLUCIÓN

El champion decide, las sombras observan



Arquitectura propuesta



MODULO 1: SCORING EN TIEMPO REAL

Dos puertas de entrada, un solo modelo



Pago en vivo
necesita respuesta en ms

gRPC: milisegundos

Analista
UI

HTTP

API REST
valida el pedido y lo reenvía

gRPC

gRPC Motor de scoring

1. Arma las features
2. Champion decide
3. Las sombras opinan

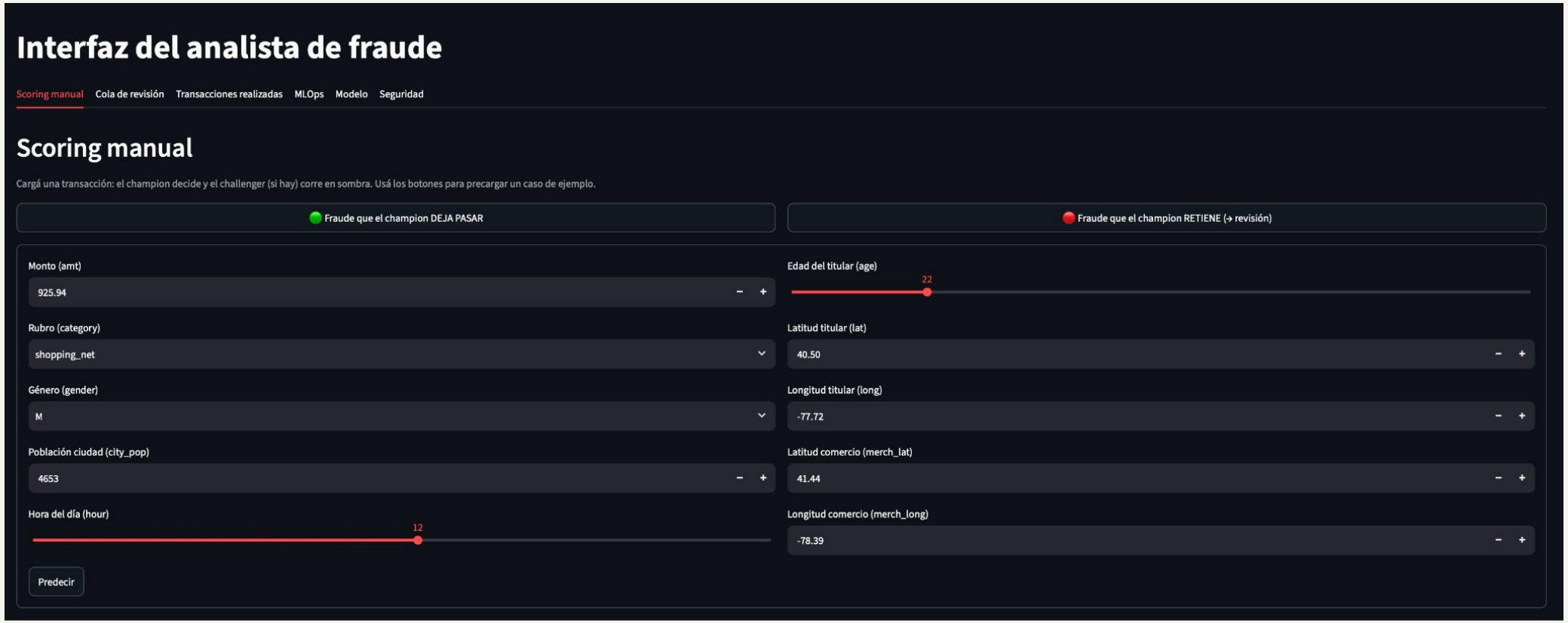
Las sombras guardan su predicción, pero no cambian el resultado.

Aprueba o retiene

El pago necesita velocidad (gRPC).

MODULO 1: SCORING EN TIEMPO REAL

UI donde hacemos “transacciones”



MODULO 2: ANALISTA Y GROUND TRUTH

Ground truths



PENDING

El champion la retiene

→

Cola de revisión

el analista aprueba o rechaza

→

`label 0 / 1`

`source=review`

SQLite

OK

El champion la aprueba

→

Denuncia post-mortem

contracargo días después

→

`label 1`

`source=post_mortem`

SQLite

Ground truth por transaction_id sirve para evaluar las sombras y vuelve como datos del próximo retrain.

MODULO 2: ANALISTA Y GROUND TRUTH

Ground truths

Cola de revisión

Transacciones retenidas (PENDING). Resolvé: aprobar = legítima · rechazar = fraude.

transaction_id	monto	prob_champion	prob_challenger	prob_federado
tx-dd793c417d5b	800	0.9212	None	None

Resolver

tx-dd793c417d5b — monto 800.0 · grocery_pos · hora 2

Aprobar Rechazar

Transacciones realizadas

Histórico de transacciones ya procesadas. Las que se dejaron pasar (aprobadas / auto-aprobadas) pueden denunciarse como fraude a posteriori (post-mortem).

transaction_id	estado	decisión	monto	prob_champion
tx-d5ea841bd8fb	REJECTED	reject	800	0.9212
tx-f9cdae20d750	OK	approve	925.94	0.1108

Detalle y acciones

› tx-d5ea841bd8fb — REJECTED — monto 800.0 · grocery_pos · hora 2

› tx-f9cdae20d750 — OK — monto 925.94 · shopping_net · hora 12

MODULO 3: REENTRENAMIENTO

Reentrenando con el feedback



DAG `fraud_pipeline`

`ingest`
dataset histórico
MinIO (S3)

→

`features`
FE + split estratificado

→

`merge_feedback`
suma los labels del analista
SQLite

→

`train`
XGBoost con balanceo de clases
XGBoost

→

`register_model`
MLflow como @challenger
MLflow
MinIO (S3)

MODULO 5: APRENDIZAJE FEDERADO

Varios bancos, un modelo, cero datos compartidos



Banco A
silo oeste, entrena local
PyTorch

Banco B
silo este, entrena local
PyTorch

solo pesos

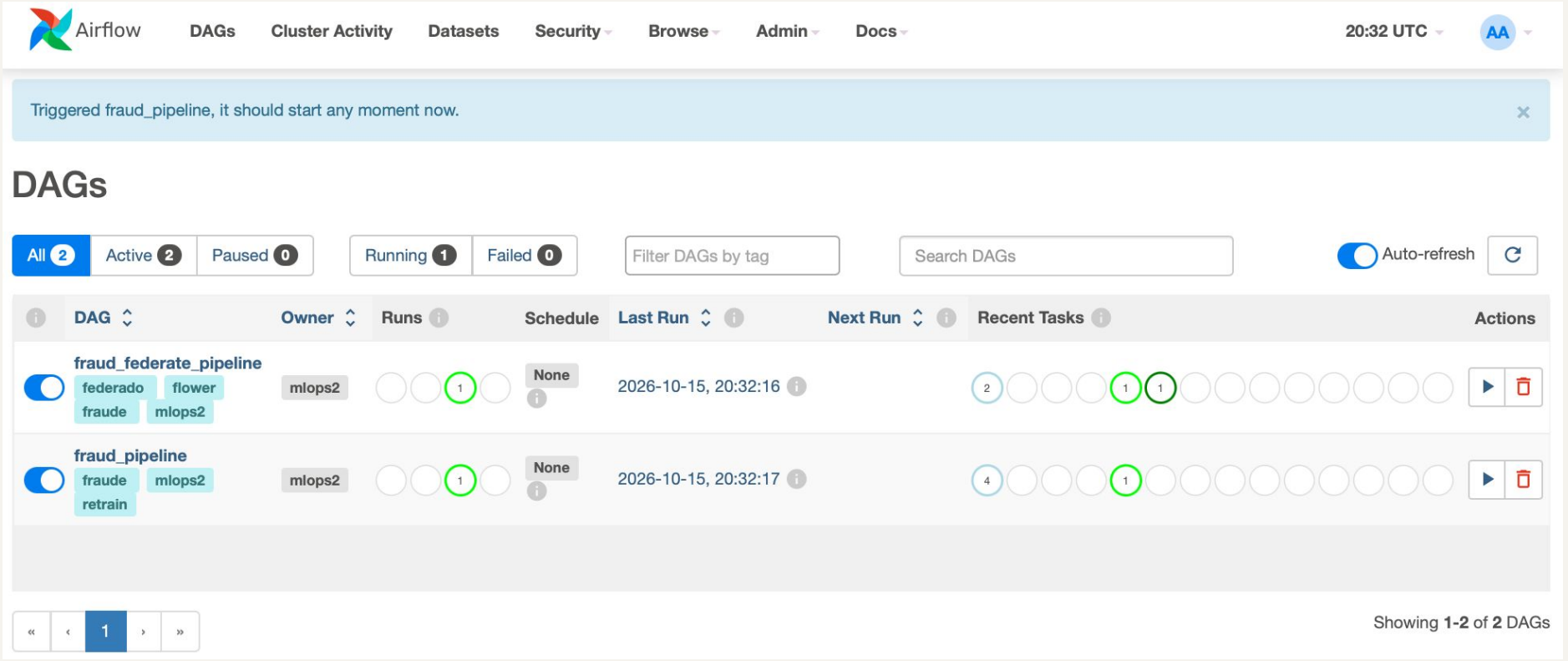
Servidor Flower
FedAvg: promedia pesos ponderando por ejemplos, 5 rondas
flwr Flower
Airflow
Docker

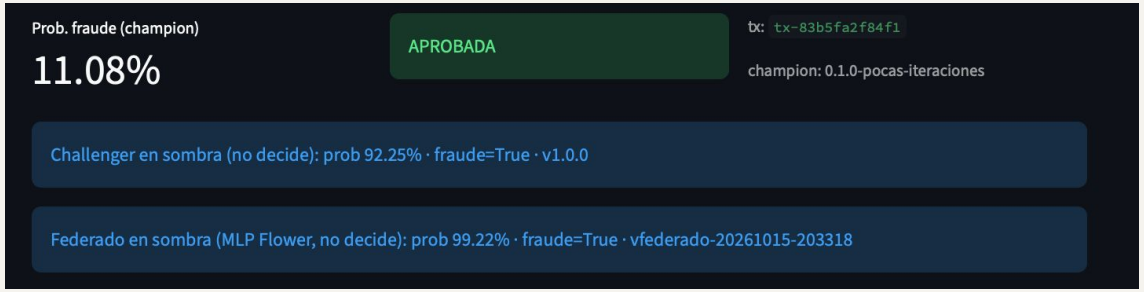
Modelo global MLP
registrado como @federated
PyTorch
MLflow

Silos no-IID
cada banco ve una región distinta, como en la realidad

¿Por qué MLP y no XGBoost?
los árboles no tienen pesos que promediar

EN VIVO: REENTRENAMIENTO





MODULO 4: REGISTRY Y DEPLOY

Promover un modelo es mover un alias

ALIASES EN MLFLOW

@champion
decide

@challenger
sombra, último retrain

@federated
sombra, modelo Flower

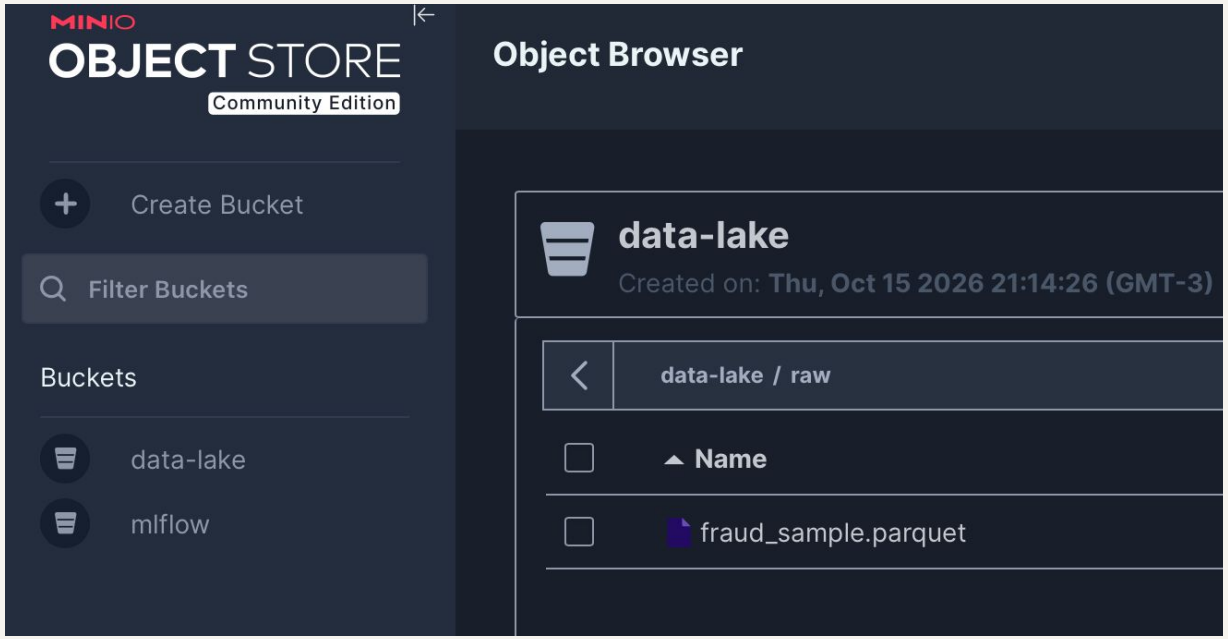


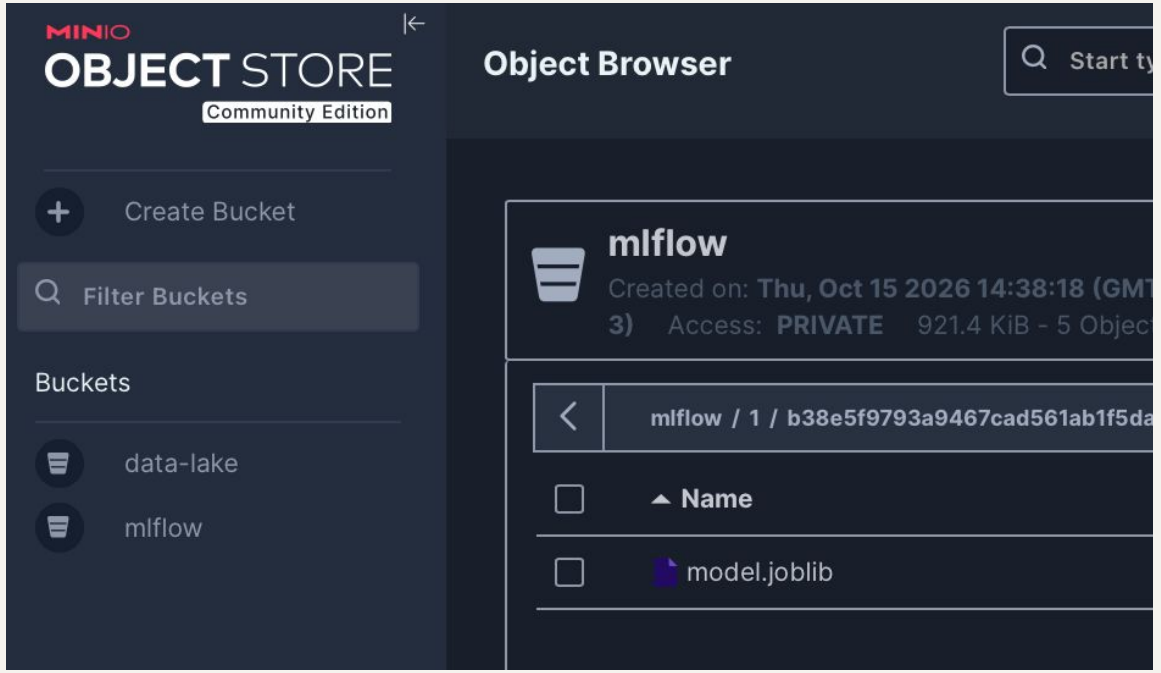
	Version	Registered at	Created by	Tags	Aliases
✓	Version 3	2026-10-15 17:34:45		Add	@ challenger
✓	Version 2	2026-10-15 17:33:19		Add	@ federated
✓	Version 1	2026-10-15 17:14:16		Add	@ champion

MODULO 4: REGISTRY Y DEPLOY

Donde guardamos los artefactos ?







MODULO 4: REGISTRY Y DEPLOY

Promover un modelo es mover un alias



CICLO DE PROMOCIÓN

1. Evaluar

⬇

2. Promover
el MLOps elige challenger o federado

⬇

3. Recarga en caliente
gRPC recarga sin rebuild

Recall de fraude

0.32
champion inicial: 5 árboles, sin balanceo

0.93
challenger: 300 árboles, con balanceo

- se promueve

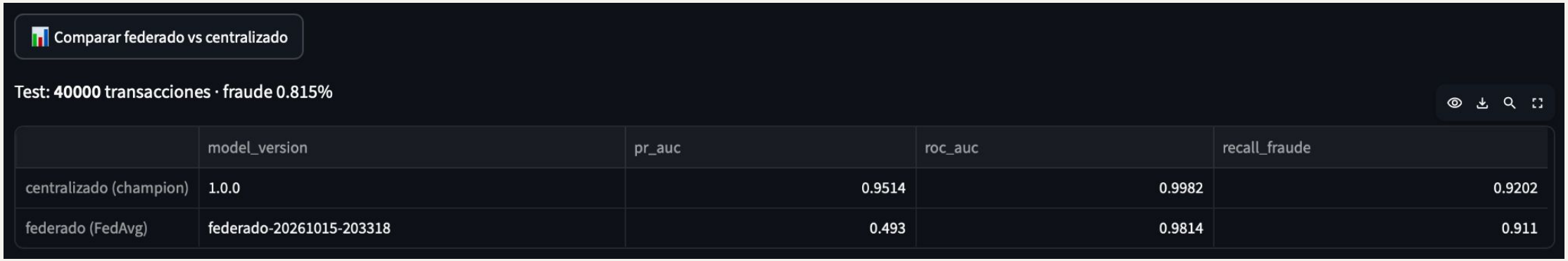
MODULO 5: RESULTADOS

El precio de la privacidad

Métrica	Federado	Centralizado
PR-AUC	0.49	0.95
Recall de fraude	0.91	0.92
ROC-AUC	0.98	0.99

49 %

de PR-AUC, sin que ningún banco comparta una transacción.



EN VIVO: REGISTRY Y EVALUACIÓN

La evidencia antes del deploy



Sembrá fraudes post-mortem para poblar la ground truth. Corré esto DESPUÉS de **Recargar modelos** (con el challenger activo) para que se registren las predicciones de ambos modelos y la evaluación tenga sentido.

🌱 Sembrar 30 fraudes (post-mortem)

Evaluar sombra vs champion

Transacciones con ground truth: **91**

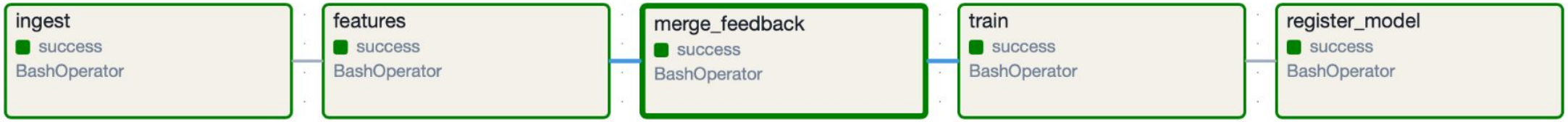
	n	precision	recall	f1	pr_auc	confusion	false_positives	fraud_amount_total	fraud_amount_captured
champion	91	1	0.2747	0.431	None	{"tp":25,"fp":0,"fn":66,"tn":0}	0	57874.07	10477.85
challenger	90	1	0.9333	0.9655	None	{"tp":84,"fp":0,"fn":6,"tn":0}	0	57074.07	56954.01
federated	90	1	0.9333	0.9655	None	{"tp":84,"fp":0,"fn":6,"tn":0}	0	57074.07	56983.11

El challenger mejora al champion (recall challenger 0.9333 vs champion 0.2747).

El federado mejora al champion (recall federado 0.9333 vs champion 0.2747).

Evaluación contra ground truth: ambas sombras superan al champion

EN VIVO: REENTRENAMIENTO



DAG fraud_pipeline / Run ▸2026-10-15, 21:24:30 UTC / Task merge_feedback

Details | Graph | Gantt | Code | Audit Log | Logs | XCom | Task Duration

(by attempts)

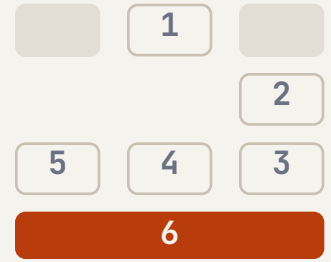
1

All Levels | All File Sources

```
59401c29c0fb
*** Found local files:
***   * /opt/airflow/logs/dag_id=fraud_pipeline/run_id=manual__2026-10-15T21:24:30.532048+00:00/task_id=merge_feedback/attempt=1.log
[2026-10-15, 21:26:31 UTC] {local_task_job_runner.py:120} ▶ Pre task execution logs
[2026-10-15, 21:26:31 UTC] {subprocess.py:63} INFO - Tmp dir root location: /tmp
[2026-10-15, 21:26:31 UTC] {subprocess.py:75} INFO - Running command: ['/usr/bin/bash', '-c', 'cd /opt/airflow/repo && python -m fraud.feedback']
[2026-10-15, 21:26:31 UTC] {subprocess.py:86} INFO - Output:
[2026-10-15, 21:26:31 UTC] {subprocess.py:93} INFO - 2026-10-15 21:26:31.786 | INFO     | fraud.config:<module>:11 - PROJ_ROOT path is: /opt/airflow/repo
[2026-10-15, 21:26:32 UTC] {subprocess.py:93} INFO - 2026-10-15 21:26:32.573 | SUCCESS  | __main__:main:86 - merge_feedback: base=120000 injected=91 total=120091
[2026-10-15, 21:26:32 UTC] {subprocess.py:97} INFO - Command exited with return code 0
[2026-10-15, 21:26:32 UTC] {taskinstance.py:441} ▶ Post task execution logs
```

MODULO 6: MONITOREO TRANSVERSAL

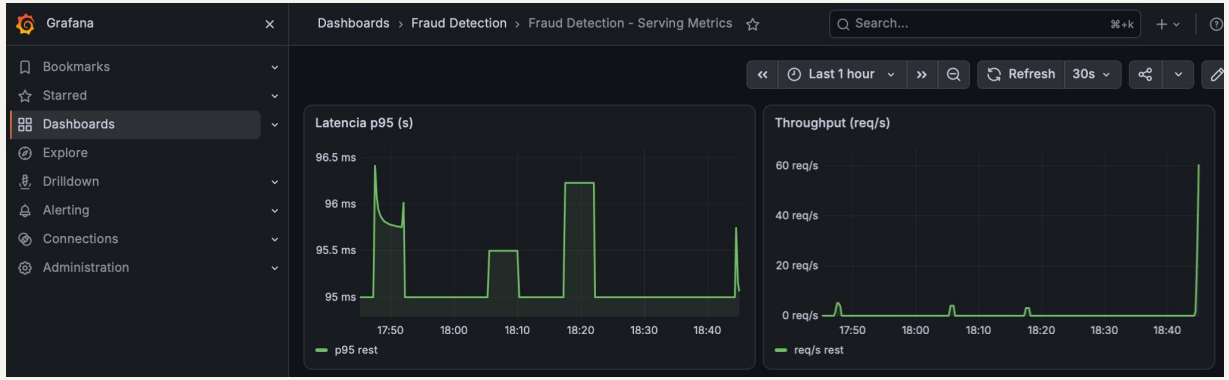
Monitoreando...



Salud del serving

latencia p95, throughput y tasa de error desde /metrics

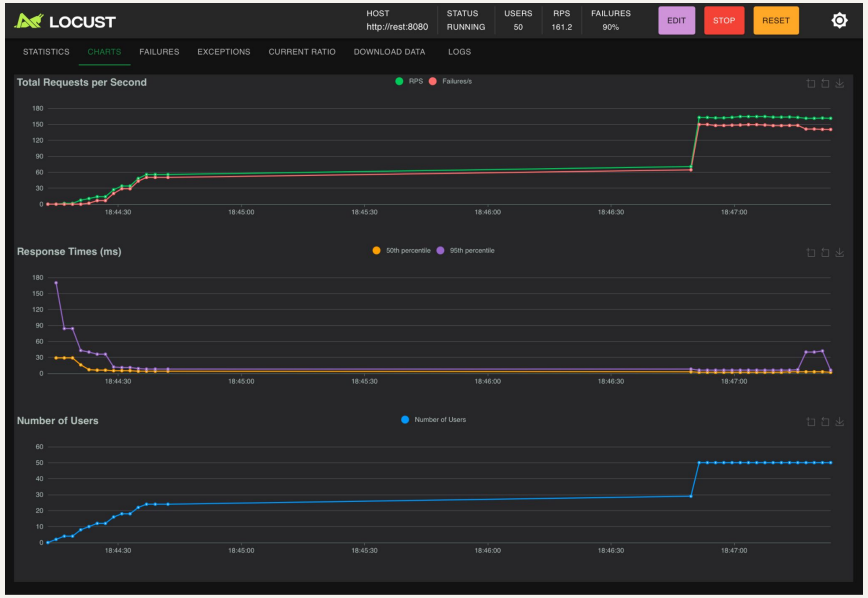
- **Prometheus**
- **Grafana**



Muchos clientes a la vez

N usuarios concurrentes con transacciones reales, p50 / p95 / p99

- **Locust**



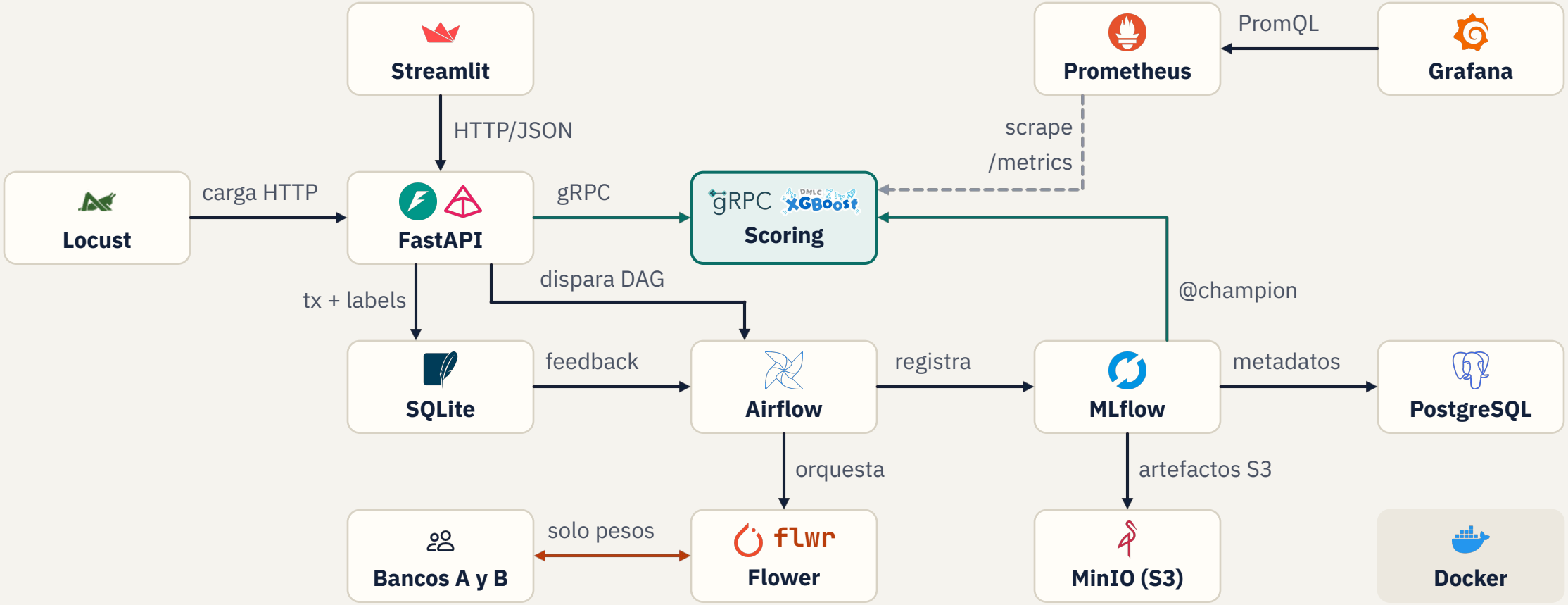
Seguridad

autenticación en REST y gRPC

- **API Key**

ARQUITECTURA CON TECNOLOGÍAS

Cómo se habla cada pieza del stack



STACK COMPLETO

Qué tecnología resuelve cada cosa

gRPC
scoring online

FastAPI
API REST

Pydantic
validación

Streamlit
UI analista

XGBoost
champion

scikit-learn
pipelines

PyTorch
MLP federado

Flower
FedAvg

Airflow
orquestación

MLflow
registry

MinIO
data lake S3

PostgreSQL
metadatos

SQLite
transacciones

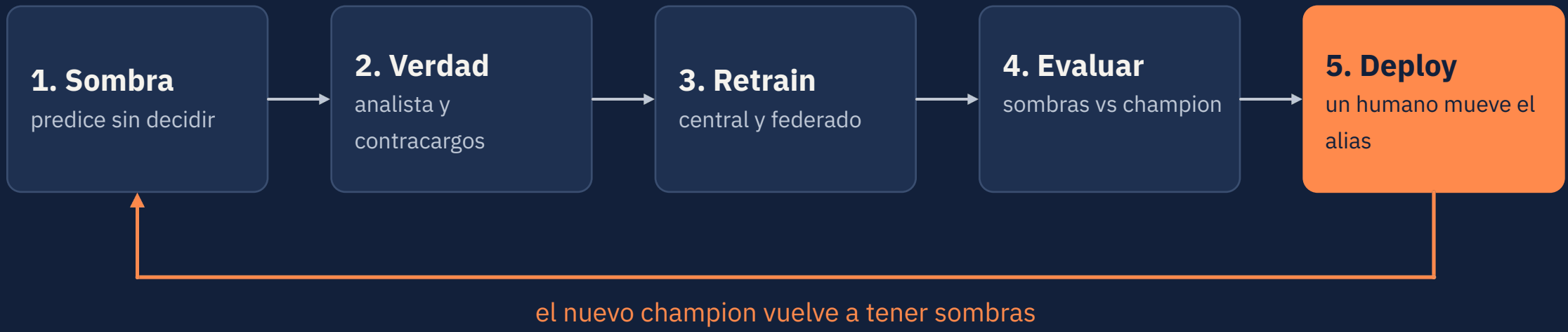
Prometheus
métricas

Grafana
dashboards

Locust
carga

Docker
contenedores

EN RESUMEN


1. Sombra
predice sin decidir
2. Verdad
analista y contracargos
3. Retrain
central y federado
4. Evaluar
sombras vs champion
5. Deploy
un humano mueve el alias
el nuevo champion vuelve a tener sombras

SEGURIDAD: ACCESO Y TRAZABILIDAD

Cada rol toca solo lo suyo, y todo queda registrado

Analista

Puede: ver la cola, aprobar, rechazar y denunciar

No puede: entrenar ni desplegar (403)

MLOps

Puede: entrenar, evaluar, promover y hacer rollback

Admin

Puede: todo lo anterior

Sin key válida

no entra ni por REST ni por gRPC

LOG DE AUDITORÍA: UNA LÍNEA POR ACCIÓN

```
{"action": "mlops.deploy", "role": "admin", "key_id": "***-123", "source": "federated", "status": "deployed"}
```

Quién, qué y cuándo, con el token enmascarado. Además, el hash SHA-256 avisa si alguien cambió el modelo.

SEGURIDAD: EL RIESGO PROPIO DEL FEDERADO

¿Y si uno de los bancos miente?

1. LIMPIO

Bancos honestos

FedAvg promedia los pesos. Línea base.

2. ATACADO

Un banco voltea sus etiquetas

El promedio arrastra al modelo global y cae el PR-AUC.

3. DEFENDIDO

Mismo atacante, servidor robusto

El modelo resiste y recupera el desempeño.

Clipping
limita cuánto puede mover cada banco al modelo

Mediana
en vez del promedio: el banco malicioso queda como outlier

Las defensas se activan por variables de entorno en el servidor Flower. Están apagadas por defecto para no alterar el benchmark federado vs centralizado.

Gracias

Repo: https://github.com/florvela/fraud-detection